EXPERIMENT 1

To understand and familiarize with the training kits and programming software used for Intel 8051 and Atmel AT89C51 microcontrollers.

i. Aim

To study and understand the architecture, functional blocks, pin configuration, and operation of Intel 8051/AT89C51 microcontroller training kits and to familiarize with the EdSim51 simulator and programming workflow.

ii. Course Outcome Mapped (COs)

CO1: Program microcontrollers to perform basic arithmetic and logical operations.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO9	PO10	PO11	PO12	PSO1

iv. Prerequisite

- Basic knowledge of digital electronics
- Understanding of microprocessor basics (ALU, registers, memory)
- Familiarity with binary and hexadecimal system
- Basic understanding of assembly language

v. Theory

Overview of 8051 / AT89C51

The 8051 is an 8-bit microcontroller with the following features:

- 4 KB ROM (on-chip)
- 128 bytes RAM
- Two 16-bit timers (Timer0, Timer1)

- 32 I/O pins (4 Ports)
- Serial communication (UART)
- 6 interrupts

Training Kit Components

- Microcontroller socket
- On-board LEDs
- 7-segment displays
- Keypad interface
- LCD module interface
- Serial port interface
- Reset circuitry
- Crystal oscillator
- Power supply (5V regulated)

EdSim51 Simulator

A virtual environment used to:

- Write and debug 8051 assembly programs
- View register, memory, and SFR states
- Observe hardware behavior (LCD, UART, keypad, LEDs)
- Simulate timing and execution cycles

vi. Software Required

- **EdSim51 Simulator**
- **8051 Microcontroller Trainer Kit**

vii. Procedure

1. Power ON the training kit and verify +5V supply.
2. Observe the layout of ports, LEDs, keypad, display modules.
3. Identify serial interface, timers, and reset circuitry.
4. Open EdSim51 → Create a new Assembly file.
5. Type a sample instruction (e.g., MOV A, #25H).
6. Assemble the program → Check for syntax errors.
7. Load the .asm file into the simulator.
8. Step through the code using single-step execution.
9. Observe changes in:
    - Accumulator

- Registers (R0–R7)
- Timers
- SFRs

10. Verify memory map and port output behavior.
11. Document observations.

viii. Program Code (Sample Test Code)

```
MOV A, #25H
MOV R0, #10H
ADD A, R0
HERE: SJMP HERE
```

ix. Observations

- Register A loaded with 25H
- Register R0 loaded with 10H
- ADD instruction executed
- A = 35H after addition
- SJMP creates an infinite loop

x. Result

Students successfully familiarized themselves with the trainer kit, its functional modules and performed basic assembly program execution using EdSim51.

xi. Outcomes Achieved

- Understood 8051 hardware layout
- Understood EdSim51 programming workflow
- Verified register-level program execution

xii. Assessment Questions

a. Pre-Lab Questions

1. What is the size of RAM in 8051?
2. Name the ports available in 8051.
3. What is the function of PSW?

b. Post-Lab Questions

1. Differentiate between MOV and MVI instructions.
2. How does the EdSim simulator display register states?
3. What is the role of the reset pin?

c. Viva-Voce Questions

1. Explain the pin configuration of 8051.
2. What is the purpose of Timers?
3. What does SFR stand for?

d. Industry/Placement-Based Questions (Matrix)

Skill Area	Industry Question	Experiment Link
Hardware Debugging	How do you verify register-level operations?	Exp 1
Embedded Testing	How do you test I/O port operations in a simulator?	Exp 1
Assembly Programming	How do you load constants into registers?	Exp 1

xiii. Faculty Assessment (Rubrics)

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 2

To perform basic arithmetic operations using a microcontroller and verify the results.

i. Aim

To write and execute 8051 assembly programs to perform arithmetic operations such as addition, subtraction, multiplication, and division.

ii. Course Outcome Mapped

CO1

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO9	PO10	PO11	PSO1

iv. Prerequisite

- Basic assembly instructions
- Register and memory organization
- Arithmetic flags (CY, AC)

v. Theory

Arithmetic Instructions

- **ADD A, Rn** → A = A + Rn
- **ADDC A, Rn** → A = A + Rn + Carry
- **SUBB A, Rn** → A = A – Rn – Carry
- **MUL AB** → A = lower byte, B = higher byte
- **DIV AB** → A = quotient, B = remainder

vi. Software Required

- EdSim51
- Training Kit

vii. Procedure

1. Write arithmetic programs in EdSim51 editor.

2. Assemble to check for syntax errors.
3. Load into simulator.
4. Observe SFR, FLAG, and register states.
5. Verify CY/AC flags.
6. Document outputs.

viii. Program Code

(a) Addition

```
MOV A, #25H
MOV R0, #12H
ADD A, R0   ; Result = 37H
HERE: SJMP HERE
```

(b) Subtraction

```
MOV A, #50H
MOV R1, #20H
SUBB A, R1
SJMP $
```

(c) Multiplication

```
MOV A, #12H
MOV B, #05H
MUL AB
SJMP $
```

(d) Division

```
MOV A, #40H
MOV B, #08H
DIV AB
SJMP $
```

ix. Observations

- ADD result = 37H
- SUBB result = 30H
- MUL result = 3C H
- DIV quotient = 08H, remainder = 00H

x. Result

Arithmetic operations successfully executed and verified using simulator.

xi. Outcomes Achieved

- Mastered arithmetic instructions
- Understood flag operations
- Verified register-level arithmetic behavior

xii. Assessment Questions

a. Pre-Lab

1. What is the function of the CY flag?
2. What is MUL AB used for?

b. Post-Lab

1. After ADD A,#0F0H, what is the CY status?
2. Explain the difference between ADD and ADDC.

c. Viva

1. How does DIV AB store output?
2. What happens when dividing by zero?

d. Industry/Placement Matrix

Skill	Question	Experiment
Arithmetic Logic	How do you verify flag changes?	Exp 2
Testing	How do you simulate arithmetic in 8051?	Exp 2

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 3

To write and execute programs for transferring data from one memory block to another memory block.

i. Aim

To demonstrate data transfer between registers, internal RAM, and external memory using MOV, MOVX, and indirect addressing modes.

ii. Course Outcome Mapped

CO1

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO9	PO10	PSO1

iv. Prerequisite

- Understanding of direct and indirect addressing
- RAM structure (00H–7FH)
- MOV/MOVX instructions

v. Theory

Data Transfer Instructions

- **MOV A, Rn**
- **MOV Rn, A**
- **MOV A, @Ri**
- **MOVX A, @DPTR** (external RAM)
- **MOVX @DPTR, A**

vi. Software Required

EdSim51, Training Kit

vii. Procedure

1. Initialize source and destination memory.
2. Set DPTR for external memory.
3. Load value from source location.
4. Store into destination block.
5. Repeat for block transfer.
6. Verify memory content.

viii. Program Code: Block Transfer

```
MOV R0, #30H  ; Source start
MOV R1, #40H  ; Destination start
MOV R2, #10   ; Count = 16 bytes

BACK: MOV A, @R0
      MOV @R1, A
      INC R0
      INC R1
      DJNZ R2, BACK
HERE: SJMP HERE
```

ix. Observations

- Data copied successfully from 30H–3FH to 40H–4FH.
- R2 decrements until zero.

x. Result

Block data transfer successfully implemented.

xi. Outcomes Achieved

- Understood addressing modes
- Implemented block transfer logic
- Mastered loop and DJNZ instructions

xii. Assessment Questions

a. Pre-Lab

1. What is indirect addressing?
2. What is the purpose of DJNZ?

b. Post-Lab

1. Why do we use R0 and R1 in this experiment?
2. Modify the code to copy 32 bytes.

c. Viva

1. Explain MOVX.
2. How do we access external memory?

d. Industry Matrix

Skill	Industry Question	Experiment
Firmware Memory Ops	How do you implement block copy?	Exp 3
Optimization	Why use DJNZ instead of LOOP?	Exp 3

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 4

To write programs implementing various arithmetic logic instructions and verify their operation.

i. Aim

To write and execute 8051 assembly programs using logical instructions such as AND, OR, XOR, CPL, CLR, RL, RR, SWAP and verify their behavior on registers and flags.

ii. Course Outcome Mapped

CO1: Program microcontrollers to perform basic arithmetic and logical operations.

iii. PO / PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO10	PO11	PSO1

iv. Prerequisite

- Understanding of boolean algebra
- Familiarity with accumulator-based operations
- Knowledge of 8051 bit-level and byte-level logical instructions

v. Theory

Logical Instructions in 8051

Instruction	Operation
ANL A, Rn	Logical AND
ORL A, Rn	Logical OR
XRL A, Rn	Logical XOR
CLR A	Clear accumulator
CPL A	Complement accumulator
RL A	Rotate accumulator left

Instruction	Operation
RR A	Rotate accumulator right
SWAP A	Swap nibbles of A

Applications

- Masking operations
- Bit extraction
- CRC operations
- Control logic in embedded systems

vi. Software Required

- EdSim51
- 8051 Training Kit

vii. Procedure

1. Load EdSim51 simulator.
2. Write separate programs for each logical instruction.
3. Assemble and correct syntax errors.
4. Observe changes in accumulator after each instruction.
5. Verify bitwise operations using memory & register window.
6. Document results.

viii. Program Code

(a) Logical AND

```
MOV A, #0F5H
MOV R0, #0A3H
ANL A, R0
SJMP $
```

(b) Logical OR

```
MOV A, #55H
MOV R1, #AAH
ORL A, R1
SJMP $
```

(c) Logical XOR

```
MOV A, #5AH
MOV R2, #3CH
XRL A, R2
SJMP $
```

(d) Complement and Clear

```
MOV A, #96H
CPL A
CLR C
SJMP $
```

(e) Rotate Left/Right

```
MOV A, #09H
RL A
                    RR A
SJMP $
```

(f) Swap Nibbles

```
MOV A, #3CH
SWAP A
SJMP $
```

ix. Observations

- AND, OR, XOR performed bitwise operations successfully.
- RL and RR rotated accumulator contents.
- SWAP exchanged upper and lower nibbles.

x. Result

Logical operations were implemented successfully and their effects verified using the simulator.

xi. Outcomes Achieved

- Understood bitwise operations in embedded systems.
- Observed accumulator-level transformations.
- Gained proficiency in logical processing instructions.

xii. Assessment Questions

a. Pre-Lab Questions

1. What is the difference between logical AND and OR?
2. What does SWAP instruction do?

b. Post-Lab Questions

1. After ANL A, #0F0H, what remains in the lower nibble?
2. How does RL differ from RLC?

c. Viva Questions

1. Define XOR with an example.
2. What is the purpose of CPL instruction?

d. Industry/Placement Matrix

Skill	Industry Question	Experiment
Logical Processing	How do you implement bit masking?	Exp 4
Hardware Control	How to toggle specific bits?	Exp 4

xiii. Faculty Assessment (Rubrics)

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 5

To generate time delays using microcontroller timer and verify the timing operations.

i. Aim

To generate software and hardware delays using Timer0/Timer1 in mode 1 and verify timing accuracy using EdSim51 simulator.

ii. Course Outcome Mapped

CO2: Implement timing and interrupt-based applications using microcontroller peripherals.

iii. PO / PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO10	PO11	PSO1	PSO2

iv. Prerequisite

- Clock frequency understanding
- Timer operation in 8051
- THx and TLx registers
- TMOD and TCON registers

v. Theory

Timer Modes

- **Mode 0:** 13-bit
- **Mode 1:** 16-bit
- **Mode 2:** 8-bit auto-reload
- **Mode 3:** Split timer

Timer Registers

- TMOD – Mode selection
- TCON – Start/Stop (TR0/TR1), Overflow (TF0/TF1)
- THx, TLx – Timer high/low byte

Delay Formula

Delay = (65536 – initial_value) × machine_cycle_time
Machine cycle = 12 / Clock Frequency

For a **12 MHz crystal → machine cycle = 1 µs**

vi. Software Required

EdSim51, Training Kit

vii. Procedure

1. Load EdSim51 simulator.
2. Configure TMOD for Timer0 Mode1.
3. Load initial value into TH0 and TL0.
4. Set TR0 = 1 to start timer.
5. Poll TF0 flag for overflow detection.
6. Verify delay using simulator stopwatch.

viii. Program Code (1 ms Delay Example)

```
MOV TMOD, #01H    ; Timer0, Mode1
MOV TH0, #0FCH    ; Load 1 ms value (65536 – 1000 = 64536 = FC18H)
MOV TL0, #018H
SETB TR0          ; Start timer
WAIT: JNB TF0, WAIT
CLR TR0
CLR TF0
SJMP $
```

Blink LED Delay Example

```
START: MOV P1, #00H
       ACALL DELAY
       MOV P1, #0FFH
       ACALL DELAY
       SJMP START

DELAY: MOV TMOD, #01H
       MOV TH0, #0FCH
       MOV TL0, #018H
```

```
      SETB TR0
AGAIN: JNB TF0, AGAIN
      CLR TR0
      CLR TF0
      RET
```

ix. Observations

- Timer overflow occurred as expected after calculated delay.
- LED blink frequency matched theoretical timing.

x. Result

Timer-based delay generation was successfully implemented and verified.

xi. Outcomes Achieved

- Understood timer configuration
- Verified delay accuracy
- Learned real-time control using timers

xii. Assessment Questions

a. Pre-Lab

1. What is the purpose of TMOD register?
2. Define machine cycle.

b. Post-Lab

1. Calculate TH0/TL0 values for 10 ms delay.
2. Why is TF0 flag important?

c. Viva

1. Differentiate Timer vs Counter mode.
2. What happens when TR0 is cleared?

d. Industry Matrix

Skill	Question	Experiment
RTOS Timing	How to generate real-time delays?	Exp 5
Device Drivers	How are timers configured in microcontrollers?	Exp 5

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 6

To write and execute programs demonstrating the use of interrupts in microcontrollers.

i. Aim

To demonstrate external and timer interrupt programming in 8051 using EdSim51, and verify interrupt service routine (ISR) execution.

ii. Course Outcome Mapped

CO3: Interface microcontrollers with display devices and implement serial communication.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO10	PO11	PSO1	PSO2

iv. Prerequisite

- Interrupt vector table
- IE (Interrupt Enable) register
- TCON (interrupt control)
- ISR structure

v. Theory

Types of Interrupts

- External Interrupt 0 (INT0)
- Timer0 Interrupt
- External Interrupt 1 (INT1)
- Timer1 Interrupt
- Serial Interrupt

Interrupt Registers

- **IE:** Enables/Disables interrupts

- **IP:** Sets priority
- **TCON:** EX0, EX1, IT0, IT1 bits

Interrupt Vector Locations

Interrupt	Vector Address
INT0	0003H
Timer0	000BH
INT1	0013H
Timer1	001BH
Serial	0023H

vi. Software Required

EdSim51, Training Kit

vii. Procedure

1. Load simulator.
2. Enable desired interrupt using IE register.
3. Set edge/level trigger using TCON.
4. Write ISR at appropriate vector address.
5. Trigger interrupt (e.g., INT0 via switch).
6. Verify ISR execution.

viii. Program Code

External Interrupt 0 Example

```
ORG 0000H
LJMP MAIN

ORG 0003H       ; INT0 ISR
INC P1
RETI

MAIN: MOV IE, #81H   ; Enable EX0
      MOV TCON, #01H ; Edge triggered
      MOV P1, #00H
```

```
HERE: SJMP HERE
END
```

Timer0 Interrupt Example

```
ORG 0000H
LJMP MAIN

ORG 000BH      ; Timer0 ISR
CPL P1.0
RETI

MAIN: MOV TMOD, #01H
    MOV IE, #82H     ; Enable Timer0 Interrupt
    SETB TR0
    SJMP $
END
```

ix. Observations

- ISR invoked upon external interrupt trigger.
- P1 toggled during Timer0 ISR execution.

x. Result

Interrupt operations executed successfully.

xi. Outcomes Achieved

- Learned interrupt vectoring
- Understood edge-triggered interrupts
- Gained experience in ISR programming

xii. Assessment Questions

a. Pre-Lab

1. What is an ISR?
2. What is the function of IE register?

b. Post-Lab

1. Differentiate between polling and interrupt.
2. Rewrite code for level-triggered INT0.

c. Viva

1. Why is RETI used instead of RET?
2. What is interrupt priority?

d. Industry Matrix

Skill	Industry Question	Experiment
Embedded ISR	Explain ISR execution flow.	Exp 6
Device Driver Dev.	How do interrupts help real-time systems?	Exp 6

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 7

To implement standard UART communication protocol using microcontroller.

i. Aim

To write and execute an 8051 assembly program to implement serial communication using UART in mode 1 and transmit/receive characters using EdSim51.

ii. Course Outcome Mapped

CO3: Interface microcontrollers with display devices and implement serial communication.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO10	PO11	PO12	PSO1	PSO2

iv. Prerequisite

- Baud rate calculation
- Timer1 auto-reload operation
- SBUF (Serial Buffer)
- SCON (Serial Control)
- TI (Transmit Interrupt) and RI (Receive Interrupt)

v. Theory

UART Communication in 8051

- Full-duplex
- Asynchronous
- 8-bit data frame
- Uses **TxD (P3.1)** and **RxD (P3.0)** pins

Important Registers

Register	Purpose
SCON	Serial mode, transmit/receive enabling
SBUF	Data register for Tx/Rx
TMOD	Timer mode selection

TH1/TL1	Baud rate reload
TCON	Start/Stop timer control

Mode 1 (8-bit UART Mode)

- Start bit
- 8-bit data
- Stop bit
- Baud rate controlled by Timer1

Baud Rate Calculation

For 11.0592 MHz crystal and 9600 baud:

Baud Rate = (2^SMOD / 32) × (Timer1 Overflow Frequency)
TH1 = FDH (−3 decimal)

vi. Software Required

- EdSim51 Simulator
- 8051 Training Kit

vii. Procedure

1. Configure SCON for mode 1 (SCON = 50H).
2. Configure Timer1 in mode 2 (TMOD = 20H).
3. Load **TH1 = 0FDH** for 9600 baud.
4. Set TR1 to start timer.
5. Load a character into SBUF for transmission.
6. Wait for TI = 1.
7. Clear TI manually.
8. For reception, wait for RI = 1 and read SBUF.

viii. Program Code

Transmit a character 'A'

```
MOV TMOD, #20H     ; Timer1, Mode2
MOV TH1, #0FDH     ; Baud rate 9600
MOV SCON, #50H     ; Mode1, enable Tx & Rx
SETB TR1           ; Start Timer1

MOV A, #"A"
```

```
MOV SBUF, A       ; Send 'A'
WAIT: JNB TI, WAIT
CLR TI
SJMP $
```

Transmit a string "HELLO"

```
ORG 0000H
MAIN: MOV TMOD, #20H
    MOV TH1, #0FDH
    MOV SCON, #50H
    SETB TR1

    MOV DPTR, #MSG

NEXT: CLR A
    MOVC A, @A+DPTR
    JZ END
    MOV SBUF, A
HERE: JNB TI, HERE
    CLR TI
    INC DPTR
    SJMP NEXT

END: SJMP END

MSG: DB "HELLO", 00H
END
```

ix. Observations

- Characters transmitted successfully on virtual terminal.
- String "HELLO" appeared on serial window.

x. Result

UART communication was successfully implemented using 8051 in mode 1.

xi. Outcomes Achieved

- Learned UART configuration

- Performed character and string transmission
- Understood serial register operations

xii. Assessment Questions

a. Pre-Lab

1. What is the purpose of TH1 in UART?
2. What is TI flag?

b. Post-Lab

1. How to modify program for 4800 baud?
2. Explain the function of SCON register.

c. Viva

1. What is the difference between synchronous and asynchronous communication?
2. How does 8051 receive a character?

d. Industry Matrix

Skill	Industry Question	Experiment
Serial Communication	How do you configure UART in embedded systems?	Exp 7
Device Integration	How do you send/receive data over UART?	Exp 7

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 8

To interface an LCD display with a microcontroller and display various characters/messages.

i. Aim

To interface a 16×2 alphanumeric LCD with 8051 and display text messages using assembly language.

ii. Course Outcome Mapped

CO3: Interface microcontrollers with display devices and implement serial communication.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO10	PO11	PO12	PSO1	PSO2

iv. Prerequisite

- Understanding of LCD pins (RS, RW, EN, D0–D7)
- Command vs data sending
- Delay routines

v. Theory

LCD Pin Functions

Pin	Purpose
RS	Command/Data select
RW	Read/Write
EN	Enable trigger
D0–D7	Data bus

LCD Modes

- 8-bit mode
- 4-bit mode

Common Commands

Command	Function
38H	8-bit, 2-line mode
0EH	Display ON, Cursor ON
C0H	Move to 2nd line
01H	Clear display

vi. Software Required

- EdSim51
- 8051 Training Board with LCD

vii. Procedure

1. Configure P2 = Data bus, P3.5 = RS, P3.6 = RW, P3.7 = EN.
2. Send initialization commands.
3. Use delay after every command or data write.
4. Write characters one-by-one to LCD RAM.
5. Verify display on simulator.

viii. Program Code

```
LCD_DATA EQU P2
RS EQU P3.5
RW EQU P3.6
EN EQU P3.7

ACALL INIT
ACALL CMD
ACALL MSG
SJMP $

; Initialize LCD
INIT:
   MOV A, #38H
   ACALL CMD
   MOV A, #0EH
   ACALL CMD
   MOV A, #01H
   ACALL CMD
```

```
   RET

; Send Command
CMD:
   CLR RS
   CLR RW
   MOV LCD_DATA, A
   SETB EN
   ACALL DELAY
   CLR EN
   RET

; Send Data
WRITE:
   SETB RS
   CLR RW
   MOV LCD_DATA, A
   SETB EN
   ACALL DELAY
   CLR EN
   RET

; Display Message
MSG:
   MOV DPTR, #TEXT
NEXT:
   CLR A
   MOVC A, @A+DPTR
   JZ DONE
   ACALL WRITE
   INC DPTR
   SJMP NEXT
DONE: RET

DELAY:
   MOV R7, #255
H1: MOV R6, #255
H2: DJNZ R6, H2
   DJNZ R7, H1
   RET

TEXT: DB "MICROCONTROLLER", 00H
```

END

ix. Observations

- LCD initialized correctly.
- Message "MICROCONTROLLER" displayed on line 1.

x. Result

LCD interfacing was successfully achieved using 8051.

xi. Outcomes Achieved

- Understood LCD timing control
- Implemented command/data operations
- Demonstrated embedded display output

xii. Assessment Questions

a. Pre-Lab

1. What is the purpose of the EN pin?
2. What command clears the LCD?

b. Post-Lab

1. Modify code to print text on second line.
2. What is the difference between RS=0 and RS=1?

c. Viva

1. Explain LCD initialization sequence.
2. What is DDRAM?

d. Industry Matrix

Skill	Question	Experiment
Display Control	How do you initialize an LCD?	Exp 8
UI Interface	How to write messages to LCD?	Exp 8

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 9

To interface a stepper motor with a microcontroller and control its operation.

i. Aim

To interface a unipolar/bipolar stepper motor with 8051 microcontroller and rotate it in clockwise and anticlockwise directions.

ii. Course Outcome Mapped

CO3

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO9	PO10	PO11	PSO1	PSO2

iv. Prerequisite

- Working of stepper motors
- Step sequences (full step / half step)
- Delay routines
- Driver IC (ULN2003 or L293D)

v. Theory

Stepper Motor Basics

- Rotates in steps
- Controlled using coil energizing sequence
- Accurate angular control

Full-Step Sequence

Step Output (P1.0–P1.3)

1 1000
2 0100
3 0010
4 0001

Clockwise Rotation

Follow sequence 1 → 2 → 3 → 4

Anticlockwise Rotation

Follow sequence 4 → 3 → 2 → 1

vi. Software Required

- EdSim51 (Stepper motor module)
- Training Kit with motor driver

vii. Procedure

1. Connect stepper motor to P1 through ULN2003.
2. Define coil energizing sequence.
3. Write program for clockwise rotation.
4. Insert delay between steps.
5. Verify rotation in simulator/hardware.
6. Modify sequence for anticlockwise rotation.

viii. Program Code

Clockwise Rotation

```
MOV P1, #00H

CW:
   MOV P1, #08H
   ACALL DELAY
   MOV P1, #04H
   ACALL DELAY
   MOV P1, #02H
   ACALL DELAY
   MOV P1, #01H
   ACALL DELAY
   SJMP CW

DELAY:
   MOV R6, #255
D1: MOV R7, #255
D2: DJNZ R7, D2
   DJNZ R6, D1
```

```
    RET
```

Anticlockwise Rotation

```
MOV P1, #00H

ACW:
    MOV P1, #01H
    ACALL DELAY
    MOV P1, #02H
    ACALL DELAY
    MOV P1, #04H
    ACALL DELAY
    MOV P1, #08H
    ACALL DELAY
    SJMP ACW
```

ix. Observations

- Motor rotated clockwise following programmed sequence.
- Delay controlled speed of rotation.

x. Result

Stepper motor was successfully interfaced and rotational control achieved.

xi. Outcomes Achieved

- Understood stepper motor sequence control
- Learned motor direction and speed control
- Applied digital control logic

xii. Assessment Questions

a. Pre-Lab

1. What is a step angle?
2. Why do stepper motors require a driver IC?

b. Post-Lab

1. Modify code for half-step sequence.
2. How does delay affect rotation speed?

c. Viva

1. What is the difference between unipolar and bipolar stepper motor?
2. Explain full-step sequence.

d. Industry Matrix

Skill	Industry Question	Experiment
Motor Control	How do you implement step control?	Exp 9
Robotics	How do you control rotation direction?	Exp 9

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 10

To interface the LM35/DHT11 temperature sensor with a microcontroller and measure temperature.

i. Aim

To interface LM35/DHT11 temperature sensor with 8051 microcontroller and measure/display temperature using ADC or digital protocol depending on sensor type.

ii. Course Outcome Mapped

CO4: Design embedded systems by interfacing sensors and actuators.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO6	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2

iv. Prerequisite

- ADC0804 interfacing (for LM35)
- Digital timing protocol (for DHT11)
- Understanding of sensor output characteristics
- LCD/UART data display

v. Theory

LM35 Sensor

- Analog temperature sensor
- 10 mV per °C output
- Requires ADC for microcontroller
- Temperature formula:

Temperature (°C) = (ADC_Value × 5V / 256) / 10mV

DHT11 Sensor

- Digital humidity + temperature sensor
- Uses single-wire protocol
- 40-bit digital frame
- Data format:

Byte	Description
1	Humidity Integer
2	Humidity Decimal
3	Temp Integer
4	Temp Decimal
5	Checksum

ADC0804

- 8-bit ADC
- Pins: CS, RD, WR, INTR, D0–D7, Vin+
- Successive approximation type

vi. Software Required

- EdSim51 Simulator
- Training Kit
- External ADC module (if using LM35)

vii. Procedure

1. Connect LM35 output to ADC0804 Vin+.
2. Connect ADC data pins D0–D7 to port P1 of 8051.
3. Connect RD, WR, INTR lines to microcontroller pins.
4. Initialize LCD (optional) for temperature display.
5. Start ADC conversion by toggling WR.
6. Wait for INTR = 0.
7. Read ADC value from Port1.
8. Convert ADC value to temperature.
9. Display temperature on LCD or UART terminal.

(For DHT11, the procedure is replaced with timing & bit-capture logic.)

****viii. Program Code**

(Example: LM35 + ADC0804 + LCD Display)**

```
ORG 0000H
ACALL INIT_LCD

MAIN:
   ACALL READ_ADC
   MOV B, #2
   DIV AB          ; Convert to °C (approx)
   MOV A, B
   ACALL DISPLAY
   SJMP MAIN

;--------- READ ADC -------------
READ_ADC:
   CLR P3.6        ; WR = 0
   SETB P3.6       ; WR = 1 → Start conversion
WAIT: JB P3.7, WAIT    ; INTR = 0?
   MOV A, P1       ; Read ADC value
   RET

;--------- LCD + Display Routines (Same as Exp 8) ----------
; Display A register value on LCD
DISPLAY:
```

```
    MOV R0, A
    MOV A, R0
    ADD A, #30H
    ACALL WRITE_LCD
    RET
END
```

ix. Observations

- ADC conversion produced values mapped to temperature.
- Temperature reading updated continuously on LCD.
- LM35 output increased on heating and decreased on cooling.

x. Result

Temperature sensor interfacing was successfully implemented and temperature measured in real time.

xi. Outcomes Achieved

- Understood analog-to-digital conversion
- Interfaced LM35 sensor
- Displayed sensor readings using embedded system

xii. Assessment Questions

a. Pre-Lab

1. What is the sensitivity of LM35?
2. Why do we use an ADC with LM35?

b. Post-Lab

1. How does ADC0804 generate digital output?
2. Modify code to show temperature in °F.

c. Viva

1. Explain DHT11 communication protocol.
2. What are the limitations of LM35?

d. Industry Matrix

Skill	Industry Question	Experiment
IoT Sensors	How do you read temperature digitally?	Exp 10
Embedded Data Acquisition	Explain ADC interfacing.	Exp 10

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 11

To interface a PIR (Passive Infrared) sensor with a microcontroller and detect motion.

i. Aim

To interface a PIR sensor with 8051 microcontroller and detect human motion, producing output via LED or buzzer.

ii. Course Outcome Mapped

CO4

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO6	PO8	PO9	PO10	PO11	PSO1	PSO2

iv. Prerequisite

- Digital input reading
- Sensor trigger levels
- Debouncing techniques
- Output control on microcontroller pins

v. Theory

PIR Sensor Basics

- Detects changes in infrared radiation
- Trigger output: **High (motion detected)**
- Settling time: typically 60 seconds
- Passive device – no radiation emitted

Applications

- Security systems
- Automatic lighting
- IoT automation

Electrical Characteristics

- Operating Voltage: 5V
- Output: Digital HIGH/LOW
- Mode: Retriggerable or non-retriggerable

vi. Software Required

- EdSim51
- PIR sensor module (in hardware)

vii. Procedure

1. Connect PIR output to P2.0 of 8051.
2. Connect LED/buzzer to P1.0.

3. Stabilize PIR sensor for 1 minute.
4. Write code to read PIR digital output.
5. If PIR=1 → Turn ON LED/Buzzer.
6. If PIR=0 → Turn OFF LED/Buzzer.
7. Simulate or verify on hardware.

viii. Program Code

```
MOV P1, #00H      ; Output port
MOV P2, #FFH      ; PIR input

MAIN: JB P2.0, MOTION
      CLR P1.0
      SJMP MAIN

MOTION:
      SETB P1.0
      SJMP MAIN
END
```

ix. Observations

- LED turned ON when PIR sensed motion.
- LED turned OFF during no motion.

x. Result

PIR sensor successfully interfaced with 8051 and motion detection achieved.

xi. Outcomes Achieved

- Understood digital sensor interfacing
- Implemented simple detection logic
- Applied microcontroller for automation application

xii. Assessment Questions

a. Pre-Lab

1. What is the use of PIR sensor?
2. Why is settling time required?

b. Post-Lab

1. Modify code to trigger alarm for 5 seconds after motion.
2. Explain how false triggers can be reduced.

c. Viva

1. What is retriggering mode?
2. Why does PIR detect only dynamic IR changes?

d. Industry Matrix

Skill	Industry Question	Experiment
IoT Automation	How do you detect human presence?	Exp 11
Security Systems	How do PIR sensors work?	Exp 11

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 12

To interface an ultrasonic sensor with a microcontroller and measure distance.

i. Aim

To interface an HC-SR04 ultrasonic sensor with 8051 microcontroller and measure distance using echo time measurement.

ii. Course Outcome Mapped

CO4

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO6	PO8	PO9	PO10	PO11	PSO1	PSO2

iv. Prerequisite

- Understanding of speed of sound
- Timing calculations
- 8051 Timer programming
- External signal capture

v. Theory

HC-SR04 Operation

1. Trigger pin: 10 µs HIGH pulse
2. Sensor emits ultrasonic burst
3. Echo pin goes HIGH for duration proportional to distance

Distance Formula

Distance (cm) = (Time_us / 58)
Distance (cm) = (Time_us × 0.017)

Timer Usage

- Timer0 in mode1 used to measure echo HIGH duration
- Using machine cycle: 1 µs (for 12 MHz clock)

vi. Software Required

- EdSim51
- Training Kit

vii. Procedure

1. Connect Trigger → P2.0
2. Connect Echo → P2.1
3. Send trigger pulse (10 µs).
4. Start Timer0 when Echo=1.
5. Stop timer when Echo=0.
6. Convert timer count to distance.

7. Display value via LCD/UART.

viii. Program Code (Simplified)

```
MOV TMOD, #01H    ; Timer0 Mode1
CLR TR0

TRIGGER:
   SETB P2.0     ; Trigger HIGH
   ACALL DELAY10
   CLR P2.0      ; Trigger LOW

WAIT_ECHO:
   JNB P2.1, WAIT_ECHO   ; Wait for ECHO HIGH

   SETB TR0              ; Start Timer

STOP:
   JB P2.1, STOP         ; Wait until ECHO goes LOW
   CLR TR0               ; Stop timer

   MOV A, TL0            ; Read timer value
   MOV B, #58
   DIV AB                ; Distance = Time/58 approx

   ACALL DISPLAY
   SJMP TRIGGER

DELAY10:
   MOV R7, #30           ; ~10 us delay
D1: DJNZ R7, D1
   RET
END
```

ix. Observations

- Timer counted echo pulse width.
- Distance value calculated and displayed correctly.

x. Result

Ultrasonic distance measurement successfully implemented.

xi. Outcomes Achieved

- Understood ultrasonic ranging principle

- Measured external physical quantity using embedded system
- Practiced time-to-distance computation

xii. Assessment Questions

a. Pre-Lab

1. What is the speed of sound?
2. Why is 10 µs trigger required?

b. Post-Lab

1. Modify code to measure distance in inches.
2. Explain why timer overflow needs handling.

c. Viva

1. What factors affect ultrasonic accuracy?
2. Why do we use Timer instead of delay loops?

d. Industry Matrix

Skill	Industry Question	Experiment
Industrial Automation	How to measure object distance electronically?	Exp 12
Robotics	How do ultrasonic sensors detect obstacles?	Exp 12

xiii. Faculty Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 13

To interface LEDs with microcontroller and implement various flowing lighting patterns.

i. Aim

To interface LEDs with 8051 microcontroller and implement flowing, running, and blinking LED patterns using delay and bit-shifting techniques.

ii. Course Outcome Mapped (COx)

CO5: Develop small-scale embedded projects by integrating multiple concepts and components.

iii. PO/PSO Mapped

PO1	PO2	PO3	PO4	PO5	PO6	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2

iv. Prerequisite

- Bitwise operations in 8051
- Timer-based delays
- LED interfacing through current-limiting resistors

v. Theory

LED Interfacing Basics

- LEDs require current-limiting resistors (typically 220–330Ω).
- Logic HIGH or LOW can turn the LED ON depending on connection.
- Running patterns are generated using bitwise shifts.

Common LED Patterns

1. **Running Light**
   00000001 → 00000010 → … → 10000000
2. **Left–Right Flow**
   00000001 → … → 10000000 → … → 00000001
3. **Blinking All LEDs**
   ALL ON → ALL OFF

vi. Software Required

- EdSim51 Simulator
- 8051 Training Kit

vii. Procedure

1. Connect LEDs to Port 1 through resistors.
2. Initialize Port 1 as output.
3. Implement various patterns using MOV/SHL/SWAP instructions.
4. Add delays to visualize patterns.

5. Observe LED behavior on hardware/simulator.

viii. Program Code

A. Running LED Pattern

```
MOV P1, #01H

RUN:
   ACALL DELAY
   RL P1
   SJMP RUN
```

B. Left–Right Flowing Pattern

```
MOV P1, #01H

FLOW:
   MOV R0, #08      ; 8 shifts left
LEFT:
   RL P1
   ACALL DELAY
   DJNZ R0, LEFT

   MOV R0, #08
RIGHT:
   RR P1
   ACALL DELAY
   DJNZ R0, RIGHT
   SJMP FLOW
```

Delay Routine

```
DELAY:
   MOV R6, #255
H1: MOV R7, #255
H2: DJNZ R7, H2
   DJNZ R6, H1
   RET
```

ix. Observations

- LEDs shifted left and right as programmed.
- Delay routine controlled speed of the pattern.

x. Result

Various LED lighting patterns were successfully implemented.

xi. Outcomes Achieved

- Understood digital output control
- Gained practice in using rotate instructions
- Implemented visual embedded applications

xii. Assessment Questions

a. Pre-Lab

1. Why is a current-limiting resistor required for LEDs?
2. What does RL instruction do?

b. Post-Lab

1. Modify code to blink alternate LEDs.
2. Implement a Knight Rider pattern.

c. Viva

1. Explain difference between RL and RLC.
2. What is multiplexing in LED display?

d. Industry/Placement Matrix

Skill	Industry Question	Experiment
Embedded Control	How do you generate visual output patterns?	Exp 13
Consumer Electronics	How are LED animations implemented?	Exp 13

xiii. Faculty Assessment (Rubrics)

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

EXPERIMENT 14

To interface seven-segment displays with microcontroller and make a time clock.

i. Aim

To interface a common cathode seven-segment display with 8051 and implement a digital clock using multiplexing and timer-based delays.

ii. Course Outcome Mapped

CO5

iii. PO/PSO Mapped

PO1–PO5	PO9–PO12	PSO1–PSO2

iv. Prerequisite

- Seven-segment display encoding
- Multiplexing technique
- Timer interrupts
- BCD to seven-segment decoding

v. Theory

Seven Segment Display Basics

Segments: a, b, c, d, e, f, g

Digits displayed by activating specific segments.

Common Cathode Display

- Low activates segment
- Patterns for digits:

Digit Hex Code (abcdefg)

0 3FH

1 06H

2 5BH

3 4FH

4 66H

5 6DH

6 7DH

7 07H

8 7FH

9 6FH

Multiplexing

- Enable one display at a time
- Display digits rapidly to appear continuous

vi. Software Required

- EdSim51
- Seven-segment interface board

vii. Procedure

1. Connect 4 seven-segment displays (HH:MM).
2. Port P0 → segment lines
3. Port P2 → digit enable lines
4. Configure Timer for 1 second interrupt.
5. Increment seconds → minutes → hours.
6. Display each digit using multiplexing.
7. Verify continuous clock output.

viii. Program Code (Simplified Clock Logic)

Lookup Table

```
SEG_TAB:
DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
```

Clock Program (Core Logic)

```
MOV TMOD, #01H        ; Timer0 Mode1
MOV P0, #00H
MOV P2, #0FH          ; All displays off

MAIN:
   ACALL DISPLAY_TIME
   SJMP MAIN

;--------------------------
; DISPLAY TIME USING MULTIPLEXING
;--------------------------
DISPLAY_TIME:
   MOV DPTR, #SEG_TAB
   ; Display HHMM (4 digits)
   MOV A, HOUR_TENS
   ACALL DISPLAY_DIGIT
   MOV A, HOUR_UNITS
   ACALL DISPLAY_DIGIT
   MOV A, MIN_TENS
   ACALL DISPLAY_DIGIT
   MOV A, MIN_UNITS
   ACALL DISPLAY_DIGIT
   RET
```

```
;-------------------------
; DISPLAY DIGIT
;-------------------------
DISPLAY_DIGIT:
   MOVC A, @A+DPTR
   MOV P0, A        ; Segment data
   ; Enable display by toggling P2.x here
   ACALL SMALL_DELAY
   RET

;-------------------------
; 1 SECOND DELAY
;-------------------------
SEC_DELAY:
   MOV TH0, #3CH     ; For 1s (approx)
   MOV TL0, #0B0H
   SETB TR0
WAIT: JNB TF0, WAIT
   CLR TR0
   CLR TF0
   RET
```

ix. Observations

- Seven-segment displays correctly displayed HH:MM.
- Digits updated every second.
- Multiplexing prevented flickering.

x. Result

Seven-segment display successfully interfaced and digital clock implemented.

xi. Outcomes Achieved

- Learned multiplexing technique
- Implemented real-time digital clock
- Mastered seven-segment encoding

xii. Assessment Questions

a. Pre-Lab

1. What is multiplexing?
2. Why is delay important in display refreshing?

b. Post-Lab

1. Modify the clock for HH:MM:SS.

2. Display countdown timer.

c. Viva

1. Difference between common anode and common cathode display?
2. Why do we use lookup tables?

d. Industry/Placement Matrix

Skill	Industry Question	Experiment
Display Systems	How do you multiplex multi-digit displays?	Exp 14
Real-time Devices	How is a clock implemented in embedded systems?	Exp 14

xiii. Faculty Assessment (Rubrics)

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor

VALUE ADDED EXPERIMENT

Mini Project: To design and implement a small project incorporating various concepts learned in previous experiments.

i. Aim
To design and develop a mini embedded system integrating sensors, actuators, microcontroller programming, displays, and communication modules learned across the lab course.

ii. Course Outcome Mapped
CO5: Develop small-scale embedded projects integrating multiple components.

iii. PO/PSO Mapped
All relevant POs (1–12) and PSOs (1–2).

iv. Prerequisite
- All previous experiments
- Basic circuit design
- Embedded C/Assembly proficiency
- Debugging skills

v. Theory
The mini-project must integrate:
- **Input device:** Sensor or switch
- **Processing:** 8051 microcontroller logic
- **Output device:** Display, actuator, or communication module
- **Control logic:** Timer/interrupts/delay
- **Power system:** Regulated 5V supply

Sample Mini Project Ideas
1. Temperature-controlled fan system
2. Smart automatic lighting using PIR/LDR
3. Distance-based obstacle alarm
4. Digital thermometer with LCD
5. Mini burglar alarm system
6. LED-based animation with speed control

vi. Software Required
- EdSim51
- Keil uVision (optional)
- 8051 Trainer

vii. Procedure (General Template)

1. Identify and define the project problem.
2. Select appropriate sensors and actuators.
3. Draw block diagram and circuit diagram.
4. Write assembly program integrating modules.
5. Simulate and verify module-by-module.
6. Test complete system on hardware.
7. Document results.

viii. Program Code (Sample: PIR-based Automatic Light System)

```
MOV P2, #FFH      ; PIR input
MOV P1, #00H      ; LED output

MAIN:
   JB P2.0, TURN_ON
   CLR P1.0
   SJMP MAIN

TURN_ON:
   SETB P1.0
   ACALL DELAY5S
   SJMP MAIN

; 5-second delay
DELAY5S:
   MOV R5, #50
D1: MOV R6, #200
D2: DJNZ R6, D2
   DJNZ R5, D1
   RET
END
```

ix. Observations

- System responded to input sensor.
- Output devices actuated correctly.
- Delay & logic worked as designed.

x. Result

A fully functional embedded mini-project was successfully designed and implemented.

xi. Outcomes Achieved

- Integrated knowledge from all experiments

- Demonstrated embedded systems design
- Learned real-world interfacing and debugging

xii. Assessment Questions

a. Pre-Lab

1. What components will your project use?
2. What is the functional goal?

b. Post-Lab

1. What challenges did you face in integration?
2. How did you debug hardware issues?

c. Viva

1. Explain the working of your project.
2. Suggest improvements for scalability.

d. Industry Matrix

Skill	Industry Question	Project Mapping
System Integration	How do you integrate multiple modules?	Mini Project
Embedded Design	Explain your system design choices.	Mini Project
Debugging	How did you test and validate your system?	Mini Project

xiii. Faculty Assessment (Rubrics)

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)	Very Poor (1)
Understanding of Kit	Complete	Good	Moderate	Low	Very Low
Understanding of Simulator	Complete	Good	Moderate	Low	Very Low
Execution Skills	No error	Minor error	Some errors	Many errors	Unable to execute
Documentation	Well structured	Good	Satisfactory	Incomplete	Poor